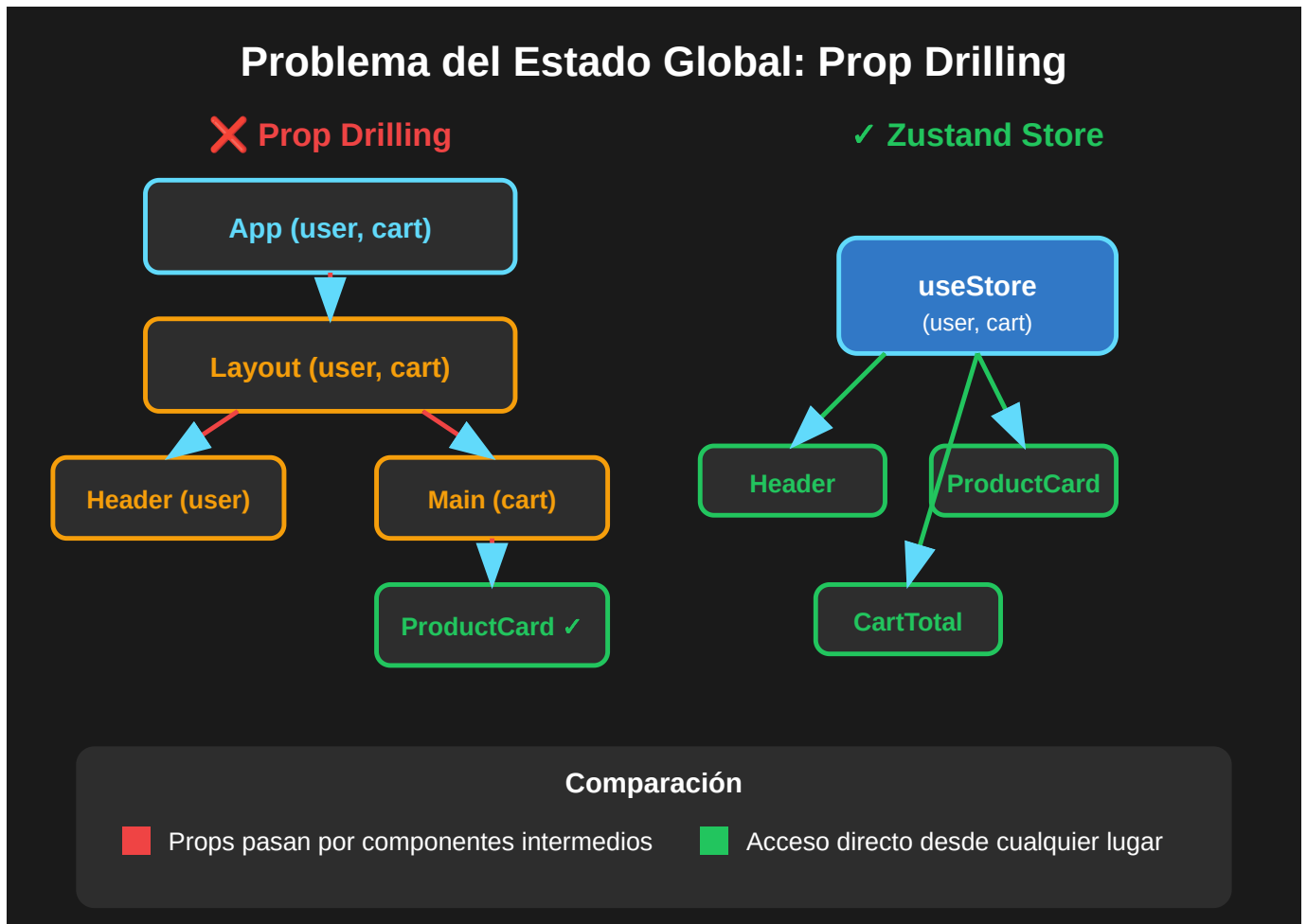


El Problema del Estado Global

Objetivos

- Comprender las limitaciones del estado local
- Identificar el problema del prop drilling
- Reconocer cuándo necesitas estado global
- Conocer las soluciones disponibles

1. Estado Local vs Estado Global



Estado Local

```
// =====  
// QUÉ: Estado que vive dentro de un componente  
// PARA: Datos que solo ese componente necesita  
// IMPACTO: Simple, predecible, encapsulado  
// =====  
  
const Counter: React.FC = () => {  
  // Estado local - solo Counter lo usa  
  const [count, setCount] = useState(0);
```

```

return (
  <div>
    <p>Contador: {count}</p>
    <button onClick={() => setCount(count + 1)}>+1</button>
  </div>
);
};

```

Estado Global

```

// =====
// QUÉ: Estado compartido entre múltiples componentes
// PARA: Datos que varios componentes necesitan
// IMPACTO: Sincronización automática, pero más complejo
// =====

// Ejemplos de estado global:
// - Usuario autenticado (Header, Sidebar, ProfilePage, etc.)
// - Carrito de compras (Navbar, CartPage, ProductCard, etc.)
// - Tema (dark/light) (App, todos los componentes)
// - Preferencias de idioma (toda la app)

```

2. El Problema: Prop Drilling

¿Qué es Prop Drilling?

Pasar props a través de múltiples niveles de componentes que no los necesitan, solo para llegar a un componente hijo que sí los usa.

```

// =====
// QUÉ: Props pasadas a través de componentes intermedios
// PARA: Llegar a un componente profundo en el árbol
// IMPACTO: Código verboso, difícil de mantener
// =====

// ❌ PROBLEMA: Prop Drilling
const App: React.FC = () => {
  const [user, setUser] = useState<User | null>(null);
  const [cartItems, setCartItems] = useState<CartItem[]>([]);

  return (
    <Layout
      user={user}
      cartItems={cartItems}
      setCartItems={setCartItems}>
      <MainContent
        user={user}
        cartItems={cartItems}
        setCartItems={setCartItems}
      />
    </Layout>
  );
};

```

```

// Layout no usa user ni cartItems, solo los pasa
const Layout: React.FC<LayoutProps> = ({
  user,
  cartItems,
  setCartItems,
  children,
}) => {
  return (
    <div>
      <Header
        user={user}
        cartItemsCount={cartItems.length}
      />
      <Sidebar user={user} />
      {children}
    </div>
  );
};

// MainContent tampoco los usa directamente
const MainContent: React.FC<MainContentProps> = ({
  user,
  cartItems,
  setCartItems,
}) => {
  return (
    <main>
      <ProductList
        user={user}
        cartItems={cartItems}
        setCartItems={setCartItems}
      />
    </main>
  );
};

// Finalmente, ProductCard sí los usa
const ProductCard: React.FC<ProductCardProps> = ({
  product,
  cartItems,
  setCartItems,
}) => {
  const addToCart = () => {
    setCartItems([...cartItems, { product, quantity: 1 }]);
  };

  return (
    <div>
      <h3>{product.name}</h3>
      <button onClick={addToCart}>Agregar al carrito</button>
    </div>
  );
};

```

Problemas del Prop Drilling

Problema	Descripción
Verbosidad	Muchas props repetidas en cada nivel
Acoplamiento	Componentes intermedios conocen datos que no usan
Mantenimiento	Agregar una prop requiere modificar toda la cadena
Refactoring	Mover componentes es complejo
Testing	Difícil testear componentes aislados

3. Solución con Context API

React incluye Context API para evitar prop drilling:

```
// =====
// QUÉ: Context API de React para estado compartido
// PARA: Evitar prop drilling básico
// IMPACTO: Solución nativa, pero con limitaciones
// =====

// Crear contexto
interface CartContextType {
  items: CartItem[];
  addItem: (item: CartItem) => void;
  removeItem: (id: number) => void;
  total: number;
}

const CartContext = createContext<CartContextType | null>(null);

// Provider
const CartProvider: React.FC<{ children: React.ReactNode }> = ({
  children,
}) => {
  const [items, setItems] = useState<CartItem[]>([]);

  const addItem = (item: CartItem) => {
    setItems((prev) => [...prev, item]);
  };

  const removeItem = (id: number) => {
    setItems((prev) => prev.filter((item) => item.id !== id));
  };

  const total = items.reduce(
    (sum, item) => sum + item.price * item.quantity,
    0,
  );

  return (
    <CartContext.Provider value={{ items, addItem, removeItem, total }}>
      {children}
    </CartContext.Provider>
  );
};
```

```

};

// Hook para usar el contexto
const useCart = () => {
  const context = useContext(CartContext);
  if (!context) {
    throw new Error('useCart debe usarse dentro de CartProvider');
  }
  return context;
};

// Uso en componente (sin prop drilling)
const ProductCard: React.FC<{ product: Product }> = ({ product }) => {
  const { addItem } = useCart();

  return (
    <div>
      <h3>{product.name}</h3>
      <button onClick={() => addItem({ ...product, quantity: 1 })}>
        Agregar
      </button>
    </div>
  );
};

```

Limitaciones de Context API

Limitación	Problema
Re-renders	Todos los consumidores se re-renderizan cuando el contexto cambia
Performance	No tiene optimización de selectores built-in
Boilerplate	Mucho código para cada contexto
DevTools	Sin herramientas de debugging dedicadas
Middleware	No soporta middleware nativamente

4. ¿Cuándo Necesitas Estado Global?

✅ Usar Estado Global

- **Datos de usuario** autenticado
- **Carrito de compras** en e-commerce
- **Tema** (dark/light mode)
- **Preferencias** de usuario
- **Notificaciones** globales
- **Estado de UI** compartido (modales, sidebars)

❌ NO Usar Estado Global

- Estado de **formularios** (mejor React Hook Form)
- Estado de **un solo componente**
- **Datos del servidor** (mejor React Query)
- Estado que **solo padres e hijos directos** comparten

5. Soluciones de Estado Global

Librería	Tamaño	Complejidad	Uso Ideal
Zustand	~1KB	Baja	Apps pequeñas-medianas
Redux Toolkit	~10KB	Media	Apps grandes, equipos grandes
Jotai	~2KB	Baja	Estado atómico
Recoil	~20KB	Media	Apps complejas con Facebook patterns
MobX	~15KB	Media	Programación reactiva

¿Por qué Zustand?

```
// =====  
// QUÉ: Zustand - librería minimalista de estado  
// PARA: Estado global simple, tipado, performante  
// IMPACTO: Menos boilerplate que Redux, más features que Context  
// =====  
  
// Comparación: Context vs Zustand  
  
// ❌ Context: ~40 líneas para un store básico  
// ✅ Zustand: ~10 líneas para lo mismo  
  
import { create } from 'zustand';  
  
interface CounterStore {  
  count: number;  
  increment: () => void;  
  decrement: () => void;  
}  
  
const useCounterStore = create<CounterStore>((set) => ({  
  count: 0,  
  increment: () => set((state) => ({ count: state.count + 1 })),  
  decrement: () => set((state) => ({ count: state.count - 1 })),  
}));  
  
// Uso  
const Counter: React.FC = () => {  
  const count = useCounterStore((state) => state.count);  
  const increment = useCounterStore((state) => state.increment);  
  
  return <button onClick={increment}>Count: {count}</button>;  
};
```

Recursos Adicionales

- [React Docs - Context](#)
- [Zustand GitHub](#)
- [State Management Comparison](#)

Checklist de Comprensión

- ☐ Entiendo la diferencia entre estado local y global
- ☐ Puedo identificar el problema de prop drilling
- ☐ Conozco las limitaciones de Context API
- ☐ Sé cuándo usar estado global
- ☐ Entiendo por qué elegir Zustand

Siguiente: [02-introduccion-zustand.md](#)